Programming SQL Server 2016
Working with Tables

Topics

- Table creation
- Basic syntax
- Data type
- Null values
- Identity column
- Unique identifier
- Sequence – new in SQL Server 2016

Homework

1. Create a table that has two columns
   Column 1: auto incrementing integer value, SQL Server to generate automatically
   Column 2: globally unique value, SQL Server to generate automatically
2. Insert 5 rows of data into the table
3. Find out the last values inserted into the column 1
4. You want to insert a row with value 250 in Column 1. Show how you do it.
5. Show me all the rows in the table.

Important Points

-
-
-

Summary

-
-
-

What is a table in database?

- In computer programming, a table is a data structure used to organize information.
- In a relational database, a table organizes the information about a single topic into rows and columns. For example, a database for a business would typically contain a table for customer information, which would store customers' account numbers, addresses, phone numbers, and so on as a series of columns.
- Each single piece of data (such as the account number) is a field in the table.
- A column consists of all the entries in a single field, such as the telephone numbers of all the customers.
- Fields, in turn, are organized as records, which are complete sets of information.

Creating a table

- In relational database management systems, all information is stored in the form of tables. So, we have to learn table creation first.
- In simple form table creation is like below:

```
CREATE TABLE table-name
(
        Column definition 1,
        Column definition 2,
        ...
        Column definition n
)
```

- A table must have at least one column
- Column definition requires at least two things
  Column-name data-type
- Assigning a data type to each column is one of the first steps to take in designing a table.
- Data types define the data value allowed for each column.
- SQL Server support a rich set data types – shown in the exhibit

Data Category	Nature of data	Supported data types
Numeric	Whole numbers	TINYINT, SMALLINT, INT, BIGINT
	Floating point	FLOAT, DOUBLE, REAL, MONEY, SMALLMONEY
Character Data	Fixed length	CHAR, NCHAR
	Variable length	VARCHAR, NVARCHAR
Binary		IMAGE, VARBINARY
Boolean		BIT
Date/Time		DATETIME, DATETIME2, SMALLDATETIME, DATE, TIME, DATETIMEOFFSET

- Above list does not list all data types, only frequently used ones
- We normally add other constraints to columns but those comes after data type:
  column-name data-type constraints
- To create a table, you must change current working database to your desired database
- The USE statement sets the current database.
- Run the following

```
USE sqlStore
GO
```

- *Make sure the sqlStote database is already created. If not create it first before running the use statement.*

Try this

```
CREATE TABLE Contacts
(
        [Id] INT,
        [Name] VARCHAR(40)
)
GO
```

- All user-defined names like Column name, table name - follows identifier rules

→ Cannot start with a number, but number can be there after first character

→ No-spaces allowed

→ Cannot use keyword

→ But if you use space or use keyword put the name in between [and]

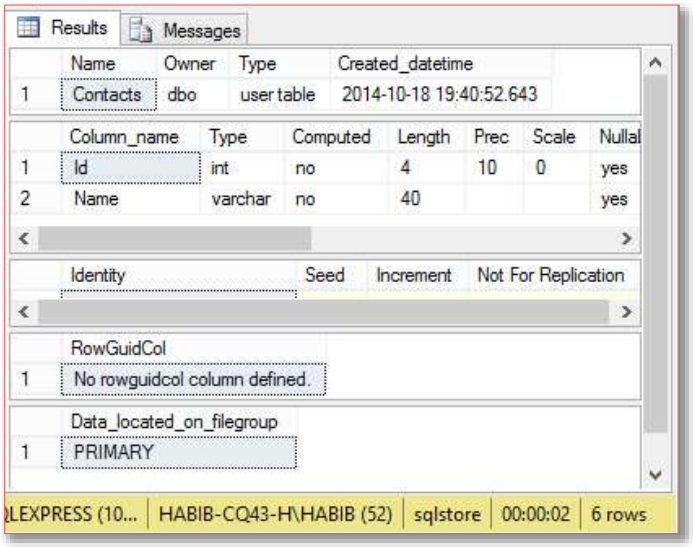
→ Example: [Order Details], [Date]

→ SQL server likes to put identifiers within []

- We created the contacts table, to view information about we can use the sp_help stored procedure
- Try this:

```
EXEC sp_help 'Contacts'
GO
```

- The result should be as show in the exhibit



- The result shows a lot of information including information about the two columns in the table.
- It shows two columns have the data type and length as we set.
- The columns got a lot of extra attributes and certainly values are set from what are the defaults defined in the database and server configuration.
- We will learn meaning of most these attributes and how to set explicitly values to them and what are the permissible values

Basic data insert and delete

- We created the table, we need to add data to and fetch data from it.
- DML in SQL allows retrieving, storing, modifying or deleting data.
- The following four statements are the basis of DML

→ SELECT – used to fetch data from a table

→ INSERT – used to store data in a table

→ UPDATE – used to modify data stored in a table

→ DELETE - used to remove data from a table

- Insert statement syntax

```
INSERT INTO table-name (column1, column2,...)
VALUES (value1, value2,...)
```

Try this

```
INSERT INTO Contacts ([Id], [Name]) VALUES (1, 'Anup')
GO
```

- All values except numeric ones must be in quotes (' ')
- Column order can different from as they are in table

Try this

```
INSERT INTO Contacts ( [Name], [Id] ) VALUES ('Foysal', 2)
GO
```

- If you put values all the column values and ordered as they are in table, you can omit column names

Try this

```
INSERT INTO Contacts VALUES (3, 'Bashar')
GO
```

- To retrieve data from a table, you have to use select statement
- Select statement syntax

```
SELECT column1, column2,.. FROM table-name
```

Try this

```
SELECT [Id], [Name] FROM Contacts
GO
```

- The result should be as shown in the exhibit

Id	Name
1	Anup
2	Faisal
3	Bashar

- You can use * wild card. It stands for all columns

Try this

```
SELECT * FROM Contacts
GO
```

- You can choose any sub-set from the columns in the table

Try this

```
SELECT [Name] FROM Contacts
GO
```

- You can change column order

```
SELECT [Name], [Id] FROM Contacts
GO
```

- we just learned very simple form of data insert and select statements
- These statements are very powerful and can be very complex in form for doing complicated and meaning tasks
- We will learn DML commands in details in later sessions

Nullabilty constraint

- After data type the most common constraint is nullabilty
- Nullability means whether the column value is required or not
- NOT NULL means the column, you must give value for this column while creating a new row in table
- NULL means you are allowed to create new row without giving value for the column
- If you do not give value for a null allowed column, the column value will be set to NULL

Try this

```
CREATE TABLE Contacts_1
(
        [Id] INT not null,
        [Name] VARCHAR(30)
)
GO
```

→ Id column is required
→ Name column is optional
→ If no value is present, it will be null
→ If you do not provide nullabilty for column, the column will be nullable by default

Try this

```
INSERT INTO Contacts_1 (Name) VALUES ('Habib')
GO
```

- You will see Error.

```
Cannot insert the value NULL into column 'Id',
table 'sqlStore.dbo.Contacts_1'; column does
not allow nulls. INSERT fails.
```

- But the following is successful, since name column allows null

```
INSERT INTO Contacts_1 ( Id) VALUES( 2 )
GO
```

- See the data

```
SELECT * Contacts_1
GO
```

- The result

Id	Name
1	NULL

- Name column has NULL value

Auto-generating unique data

- SQL server allows two way of generating auto-generating unique values for rows in your table
→ IDENTITY for generating auto-incrementing numeric values
→ GUID (uniqueidentifier) for generating globally unique values
→ SQL Server 2012 introduced a new way of inserting auto-generating unique data – sequence
→ A sequence generates a sequence of numeric values according to the specification.

IDENTITY

- it creates auto incrementing values in columns
- Identity (n, i) numbering starts from n (seed value) and increments by i at each new value creation

Try this

```
CREATE TABLE Contacts_2
(
        [Id] INT IDENTITY(1, 1) not null,
        [Name] VARCHAR (45) null
)
GO
```

- Insert some data

```
INSERT INTO Contacts_2(name ) VALUES ( 'Anis')
INSERT INTO Contacts_2(Name ) VALUES ('Amin')
INSERT INTO Contacts_2(Name ) VALUES ('Babul')
GO
```

- See the rows

```
SELECT * FROM Contacts_2
GO
```

- The result

Id	Name
1	Anis
2	Amin
3	Babul

- A table can have only one identity column
- You cannot give explicit value for identity column
- The following will raise error

```
INSERT INTO person_2 VALUES (4, 'Kamrul')
GO
```

- If delete a row, its identity column value will never be reused
- Delete a row

```
DELETE person_2 WHERE Id = 2
GO
SELECT * FROM person_2
GO
```

- How can we reuse 2 in Id column? There is a way

Inserting explicit value in identity column

- To do this, you have to set on the IDENTITY_INSERT ON for the table, it is OFF by default
- We do this in a tricky way
  1. Set IDENTITY_INSERT ON for the table
  2. Insert explicit value in identity column, you must specify the column name
  3. Set IDENTITY_INSERT OFF for the table

Try this

```
SET IDENTITY_INSERT Contacts_2 ON
INSERT INTO Contacts_2([Id],[Name])
VALUES (2, 'Jamil')
SET IDENTITY_INSERT Contacts_2 OFF
GO
SELECT * FROM person_2
GO
```

- See the result

Id	Name
1	Anis
2	Jamil
3	Babul

Retrieving the last identity inserted

- There are three ways

→ @@IDENITY- returns the last identity value inserted in the current session across all scope

Try this

```
INSERT INTO Contacts_2 (Name )  VALUES( 'Badrul')
GO
SELECT @@IDENTITY
GO
```

- IDENT_CURRENT - returns the last identity value inserted in a table in any session and any scope

Try this

```
INSERT INTO Contacts_2 VALUES ( 'Fahad')
GO
SELECT IDENT_CURRENT ( 'Contacts_2')
GO
```

- SCOPE_IDENTITY will be discussed later

Unique Identifier

- SQL server has a data type uniqueidentifier
- This data type is used to generate GUID (Globally Unique Identifier) which is unique globally
- SQL server provides built-in function NEWID(), which can generate GUID value for you

Try this

```
CREAT TABLE products
(
        id UNIQUEIDENTIFIER NOT NULL,
        name VARCHAR (30)
)
GO
```

- Insert some data

```
INSERT INTO products( id, name)
VALUES (NEWID(), 'P 1')
INSERT INTO products(id, name)
VALUES (NEWID(), 'P 2')
GO
```

- See the rows

```
SELECT * FROM products
GO
```

id	name
312FD39E-5C07-44AC-90CC-16A9DBDDDC68	P 1
58177A24-CAA8-4239-976B-68B991060E96	P 2

Sequence

- SEQUENCE is one of the new feature introduced in SQL Server 2012.
- Sequence is a user-defined object and as name suggests it generates sequence of numeric values according to the properties with which it is created.
- It is similar to Identity column, but there are many differences between them. Some of the major differences between them are:

→ Sequence is used to generate database-wide sequential number, but identity column is tied to a table.

→ Sequence is not associated with a table.

→ Same sequence can be used in multiple tables.

→ It can be used in insert statement to insert identity values, it can also be used in T-SQL Scripts.

Try this

```
CREATE SEQUENCE [DBO].[SequenceExample]
AS  INT
 START WITH 1
 INCREMENT BY 1
GO
```

Using Sequence in an Insert Statement

```
CREATE TABLE Employee(ID INT,Name
VARCHAR(100))
GO
```

- INSERT RECORDS to the Employee table with Sequence object

```
INSERT INTO dbo.Employee VALUES
(NEXT VALUE FOR DBO.SequenceExample,
'BASAR'),
(NEXT VALUE FOR DBO.SequenceExample,
'SALAM'),
(NEXT VALUE FOR DBO.SequenceExample,'ALAM')
GO
```

- Check the records inserted in the table

```
SELECT * FROM Employee
GO
```

- Result

ID	Name
1	BASAR
2	SALAM
3	ALAM

How to get the current value of the Sequence

```
SELECT Current_Value
FROM SYS.Sequences
WHERE name='SequenceExample'
GO
```

- Result

Current_Value

3

Getting Next Sequence Value in A SELECT Statement

```
SELECT (NEXT VALUE FOR DBO.SequenceExample)
AS SequenceValue
GO
```

- Result

```
4
(1 row(s) affected)
```

Re-Setting the Sequence Number

```
ALTER SEQUENCE DBO.SequenceExample
RESTART WITH 1 ;
GO
```

- Verify whether sequence number is re-set

```
SELECT (NEXT VALUE FOR DBO.SequenceExample)
                AS SequenceValue
GO
```

- Result

```
SequenceValue
————
1
```